Lecture Notes on Recurrent Neural Networks: Concepts, Diagrams, Mathematics, and Examples

Prof. Dr. Olushina Olawale Awe
PH Ludwigsburg University, Germany

1 Introduction to Recurrent Neural Networks

Recurrent Neural Networks (RNNs) are neural architectures specifically designed to process **sequential** or **temporal** data. Unlike feed-forward networks that assume independence between inputs, RNNs maintain a hidden internal memory that evolves over time.

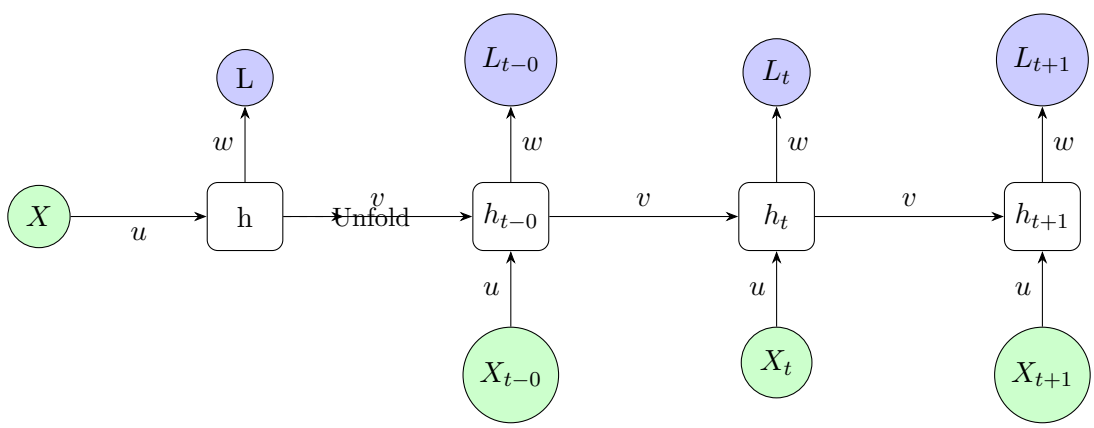
Typical applications include:

- Natural Language Processing (NLP)
- Speech recognition
- Machine translation
- Time-series forecasting
- Sequential decision-making

The key idea is that information from previous steps can be passed forward through the hidden state. These brief notes provide a comprehensive introduction to RNNs, from conceptual diagrams to mathematical representations and numerical examples. The diagrams and flowcharts here will allow you to visually understand sequence structures and the design of recurrent architectures.

2 RNN Unfolding Diagram

The figure below shows a standard RNN diagram showing how a single recurrent cell unrolls across time.



Interpretation of the Diagram

- The RNN cell (h) is **reused** for each time step.
- Inputs X_t are fed one at a time.
- Each step produces a hidden state and an output.

- Hidden state flows forward to preserve memory.

Mathematically:

$$h_t = \tanh(W_x x_t + W_h h_{t-1})$$

Explanation of the RNN Unfolding Diagram

1. The diagram illustrates how a recurrent neural network (RNN) cell is applied repeatedly across time. When the network is "unfolded," the same recurrent cell appears at each time step, reflecting the reuse of parameters.

2. Each green circle X_t represents an input at time step t. These inputs are fed sequentially into the network.

3. Each rectangular node h_t denotes the hidden state, which carries information from both the current input and all previous hidden states, acting as the memory of the RNN.

4. The arrows labeled u, v, and w correspond to learned weight matrices:

   - u: input-to-hidden weights,
   - v: hidden-to-hidden (recurrent) weights,
   - w: hidden-to-output weights.

5. The blue nodes L_{t-0}, L_t, and L_{t+1} represent outputs or loss values computed at each time step from the hidden state. These may be the predicted outputs y_t or loss terms used for training.

6. The dashed "Unfold" arrow indicates how the recurrent computation expands across time, revealing the repeated application of the same cell.

7. Information flows forward through the hidden states, enabling the network to encode temporal dependencies across the sequence.

8. This unfolded representation also clarifies how gradients propagate during training (backpropagation through time), since each hidden state depends on all previous states.

3 Sequence Input–Output Types in RNNs (Mathematics, Diagrams, Examples)

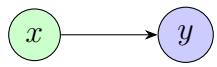
This section presents the mathematical forms, diagrams, and numerical examples of the four major sequence input–output mappings used in RNN architectures.

3.1 1. One-to-One Mapping

Equations

A single input produces a single output:

$$y = g(Wx + b)$$



Numerical Example

Let:

$$W = 3, \quad b = -1, \quad x = 2$$

Then:

$$y = 3(2) - 1 = 5$$

3.2 One-to-One RNN Mapping vs Feedforward Neural Network

At first glance, a one-to-one mapping in a recurrent neural network (RNN) looks similar to a standard feedforward neural network (FFNN), since in both cases a single input is mapped to a single output. However, the underlying architectures are fundamentally different.

In a feedforward neural network, the computation is purely feedforward:

$$h = f(Wx + b), \qquad y = g(Vh + c),$$

where x is the input, h is a hidden representation, y is the output, and f, g are activation functions. There are no recurrent connections and no notion of temporal dependence: each input example is treated as an independent observation.

In contrast, an RNN is designed to process sequences and therefore maintains a hidden state that can carry information across time. Even in the special case of a one-to-one mapping (sequence length $T = 1$), the RNN update has the form

$$h_0 = h_{\text{init}}, \qquad h_1 = f(W_x x_1 + W_h h_0 + b_h), \qquad y_1 = g(W_y h_1 + b_y),$$

where h_0 is the initial hidden state and W_h represents recurrent (hidden-to-hidden) weights. Although for $T = 1$ the temporal dimension is not exploited, the model still possesses a recurrent structure and the capacity to use past information in longer sequences.

Thus, while both FFNN and one-step RNNs implement a one-input–one-output mapping, the FFNN has no mechanism for memory across time, whereas the RNN is inherently a sequential model whose architecture is capable of capturing temporal dependencies when the input length exceeds one.

3.3 2. One-to-Many (Sequence Generation)

Equations

Initialization:

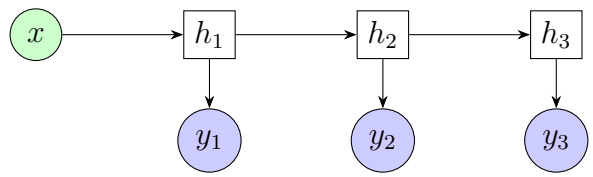
$$h_0 = f(W_{in}x + b_{in})$$

Recursive generation:

$$h_t = f(W_h h_{t-1} + b_h), \quad t = 1, \ldots, T$$

Output at each time step:

$$y_t = g(W_y h_t + b_y)$$



Numerical Example

Let:

$$h_0 = \tanh(2) = 0.964$$
$$W_h = 0.5, \quad b_h = 0$$

Then:

$$h_1 = \tanh(0.5 \cdot 0.964) = 0.446$$
$$h_2 = \tanh(0.5 \cdot 0.446) = 0.221$$
$$h_3 = \tanh(0.5 \cdot 0.221) = 0.110$$

Output:

$$y_t = h_t$$

3.4 3. Many-to-One Mapping (Sequence Classification)

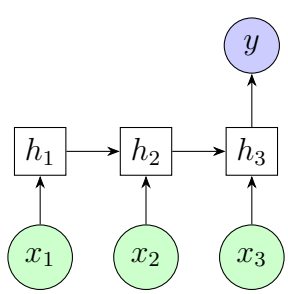
Equations

Hidden recurrence:

$$h_t = f(W_x x_t + W_h h_{t-1} + b_h)$$

Final output:

$$y = g(W_y h_T + b_y)$$



Numerical Example

Given:

$$x_1 = 1,\ x_2 = 2,\ x_3 = 3, \quad W_x = 1,\ W_h = 0.5$$

$$h_1 = \tanh(1) = 0.761$$
$$h_2 = \tanh(2 + 0.5 \cdot 0.761) = \tanh(2.3805) = 0.983$$
$$h_3 = \tanh(3 + 0.5 \cdot 0.983) = \tanh(3.4915) = 0.998$$

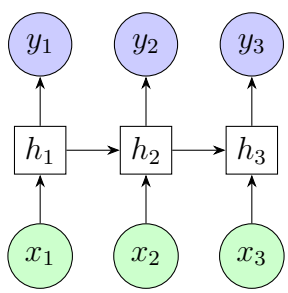
Final output:

$$y = h_3 = 0.998$$

3.5 4. Many-to-Many (Aligned)

Equations

$$h_t = f(W_x x_t + W_h h_{t-1} + b_h)$$
$$y_t = g(W_y h_t + b_y)$$



Numerical Example

Let:

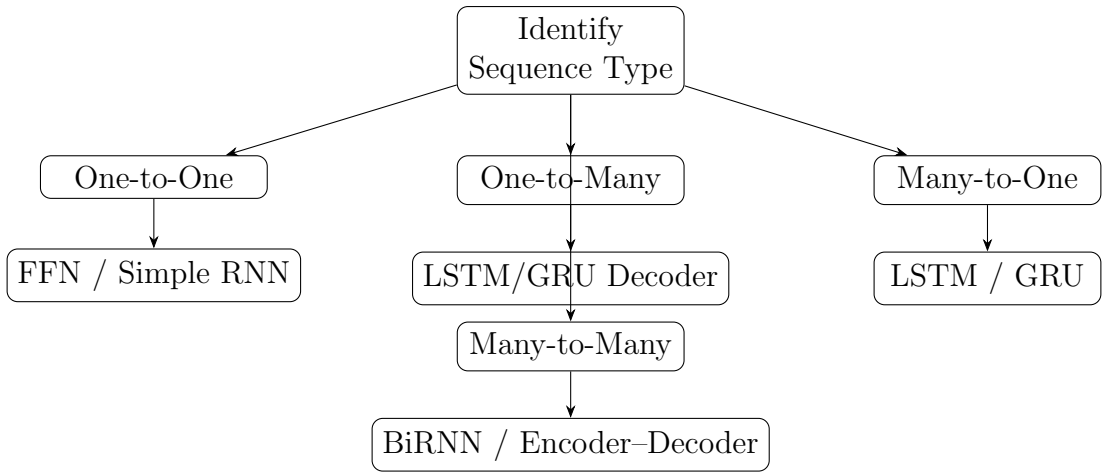
$$x_1 = 1,\ x_2 = 0,\ x_3 = 2, \quad W_x = 1,\ W_h = 0.2$$

$$h_1 = \tanh(1) = 0.761$$
$$h_2 = \tanh(0 + 0.2 \cdot 0.761) = 0.150$$
$$h_3 = \tanh(2 + 0.2 \cdot 0.150) = \tanh(2.03) = 0.966$$

Outputs:

$$y_1 = h_1, \quad y_2 = h_2, \quad y_3 = h_3$$

4 Flowchart: Choosing the Right RNN Architecture



—

5 RNN Cell Types

5.1 Vanilla RNN

$$h_t = \tanh(W_x x_t + W_h h_{t-1}) \quad \text{and} \quad y_t = W_y h_t$$

5.2 LSTM

$$f_t = \sigma(W_f x_t + U_f h_{t-1})$$
$$i_t = \sigma(W_i x_t + U_i h_{t-1})$$
$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1})$$
$$c_t = f_t c_{t-1} + i_t \tilde{c}_t$$
$$h_t = o_t \tanh(c_t)$$

5.3 GRU

$$z_t = \sigma(W_z x_t + U_z h_{t-1}), \qquad r_t = \sigma(W_r x_t + U_r h_{t-1})$$
$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t h_{t-1}))$$
$$h_t = (1 - z_t)\tilde{h}_t + z_t h_{t-1}$$

—

6 Numerical Examples

6.1 Example 1: Vanilla RNN

Given:

$$W_x = 0.5,\ W_h = 0.4,\ h_0 = 0,\ x_1 = 1,\ x_2 = 2$$

$$h_1 = \tanh(0.5) \approx 0.462$$
$$h_2 = \tanh(1.1848) \approx 0.829$$

—

6.2 Example 2: LSTM

Given:

$$f_t = 0.746,\ i_t = 0.659,\ o_t = 0.679,\ \tilde{c}_t = 0.731$$

$$c_t = 0.746(0.5) + 0.659(0.731) = 0.855$$

$$h_t = 0.679 \tanh(0.855) = 0.471$$

—

6.3 Example 3: GRU

$$z_t = 0.664, \quad r_t = 0.627, \quad \tilde{h}_t = 0.678$$
$$h_t = 0.336(0.678) + 0.664(0.4) = 0.493$$

7 Relationships and Similarities Between RNN Sequence Mappings and RNN Cell Types

Recurrent neural networks (RNNs) can operate in several input–output configurations, such as one-to-one, one-to-many, many-to-one, and many-to-many mappings. These mappings describe how sequences enter and leave the network, while the internal RNN *cell type* (Vanilla RNN, LSTM, GRU) determines how information is processed and stored over time. Although mappings and cell types serve different purposes, they interact in systematic and meaningful ways.

1. One-to-One Mapping and RNN Cells

A one-to-one mapping processes a single input and produces a single output. Architecturally, this mirrors a feedforward neural network (FFNN), but the underlying cell remains recurrent:

$$h_1 = f(W_x x_1 + W_h h_0), \qquad y_1 = g(W_y h_1).$$

Even though the recurrence is not exploited for $T = 1$, the RNN, LSTM, or GRU can be used, and all reduce to a single-step transformation.

Similarity Across Cell Types. For a single time step, the difference between Vanilla RNN, LSTM, and GRU is minimal because temporal dynamics do not accumulate. The gated mechanisms of LSTM and GRU collapse into simple weighted combinations because the previous hidden or cell state contains no prior sequence.

Relationship. One-to-one is the **degenerate case** of all RNN architectures—it shows the base nonlinear transformation of each cell type without temporal context.

2. One-to-Many Mapping and RNN Cells

A one-to-many mapping begins with a single input (or context vector) and generates a sequence of outputs:

$$h_0 = f(W_{in} x), \qquad h_t = f(W_h h_{t-1}), \quad y_t = g(W_y h_t).$$

Relationship to Cell Types. Here the recurrent structure becomes essential. All RNN cell types must repeatedly apply their transition functions:

- Vanilla RNN: repeated tanh transforms.
- LSTM: repeated gating of cell state c_t.
- GRU: repeated gated update of h_t.

Similarity Across Cell Types. All follow the same computational *pattern*:

$$\text{(initial state)} \rightarrow \text{repeated transitions} \rightarrow \text{output sequence.}$$

The difference lies only in how memory is updated (simple recurrence vs gated recurrence).

3. Many-to-One Mapping and RNN Cells

In a many-to-one mapping, a full input sequence produces a single output:

$$h_t = f(W_x x_t + W_h h_{t-1}), \qquad y = g(W_y h_T).$$

Relationship to Cell Types. This mapping is used for sequence classification tasks, where the final hidden state represents the entire sequence. Vanilla RNN, LSTM, and GRU implement this similarly:

- LSTM passes a refined memory state c_T into h_T.
- GRU passes an accumulated gated state.
- Vanilla RNN passes a nonlinear compressed summary.

Similarity Across Cell Types. All cell types aim to encode an input sequence into a **single vector representation**. The only difference is the mechanism used for storing long-range information: Vanilla RNN struggles, GRU improves, LSTM excels.

4. Many-to-Many Mapping and RNN Cells

There are two variants:

Aligned many-to-many (one input per output):

$$h_t = f(W_x x_t + W_h h_{t-1}), \qquad y_t = g(W_y h_t).$$

Encoder–Decoder many-to-many:

$$h_t^{enc} = f(W_x x_t + W_h h_{t-1}), \qquad s_t^{dec} = f(W_s s_{t-1} + W_y y_{t-1}).$$

Relationship to Cell Types. All RNN cell types support both forms:

- Vanilla RNN provides basic recurrence.
- GRU and LSTM provide deeper temporal modeling via gates.
- Encoder–decoder structures especially benefit from LSTM's ability to retain long-term memory.

Similarity Across Cell Types. The mapping structure is identical across cell types; only the internal recurrence differs. All behave as sequence transducers, but LSTM and GRU maintain information more effectively.

Unified Perspective: How Mappings Relate to Cell Types

The relationship between RNN mappings and RNN cells may be summarized as follows:

1. **Mappings describe the external structure** of the task (how many inputs and outputs).
2. **Cell types describe the internal mechanism** for processing and storing information.

3. Any mapping can use any RNN cell type; the mapping is independent of the cell.

4. Vanilla RNN, GRU, and LSTM differ only in how they compute h_t, not in the mapping itself.

5. The more complex the sequence dependency, the more beneficial gated cells (GRU, LSTM) become.

Thus, the mapping defines the *shape of the computation*, while the cell type defines the *quality of memory and temporal reasoning* within that computation.

8 Comparison of RNN Gate Types and Their Mathematical Derivations

Recurrent neural network (RNN) architectures differ in how they control information flow across time. While the Vanilla RNN has no gating mechanism, the LSTM and GRU introduce learned gates that regulate memory, enabling them to handle long-term dependencies.

Comparison of Gate Types Across RNN Cells

RNN Cell Type	Number of Gates	Gate Names
Vanilla RNN	0	None
GRU	2	Update (z_t), Reset (r_t)
LSTM	3	Forget (f_t), Input (i_t), Output (o_t)

8.1 Mathematical Derivation of LSTM Gates

The LSTM augments the recurrent neural network with gating mechanisms that allow it to retain, update, or discard information in a controlled manner. Given the input x_t, previous hidden state h_{t-1}, and previous cell state c_{t-1}, the LSTM performs the following computations.

1. Forget Gate

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$$

The forget gate produces values in $(0, 1)$ and determines which parts of the previous cell state should be retained. A value near 1 keeps information; a value near 0 removes it.

2. Input Gate

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$$

The input gate decides which components of the candidate memory $\tilde{c}_t$ will be added to the cell state.

3. Candidate Cell State

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c)$$

This candidate vector contains new potential information to write into the cell. Its values range from -1 to 1.

4. Updated Cell State

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

Here the forget gate scales the previous memory, and the input gate modulates the new memory. This linear recurrence allows gradient flow across many time steps, solving the vanishing gradient problem.

5. Output Gate

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$$

The output gate determines which parts of the internal memory are exposed to the hidden state.

6. Final Hidden State

$$h_t = o_t \odot \tanh(c_t)$$

Thus the LSTM computes hidden activation by filtering the cell state through the output gate.

Interpretation.

- Forget gate: "Which old information should I erase?"
- Input gate: "Which new information should I add?"
- Cell state: long-term memory
- Output gate: "Which part of the memory should affect the output?"

8.2 Mathematical of GRU Gates

The GRU simplifies the LSTM by merging the cell and hidden states into a single vector and reducing the number of gates.

Given input x_t and previous hidden state h_{t-1}:

1. Update Gate

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$$

This gate determines how much of the previous hidden state is carried forward:

$$z_t \approx 1 \Rightarrow \text{keep old memory}$$

$$z_t \approx 0 \Rightarrow \text{replace with new candidate}$$

2. Reset Gate

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$$

The reset gate controls how strongly the previous hidden state influences the candidate state:

$$r_t \approx 0 \Rightarrow \text{ignore old memory}$$

3. Candidate Hidden State

$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h)$$

If r_t is small, the GRU "forgets" the past entirely when generating the candidate.

4. Final Hidden State

$$h_t = (1 - z_t)\tilde{h}_t + z_t h_{t-1}$$

This convex combination blends old memory with new information, modulated by the update gate.

Interpretation.

- Update gate: retains or discards past information.
- Reset gate: controls influence of past memory when creating new information.
- Unlike LSTM, there is no separate cell state; memory is held entirely in h_t.

The GRU actually simplifies the LSTM by merging the cell state and hidden state and using only two gates.

Update gate

Controls how much of the previous hidden state to keep.

Reset gate

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$$

Controls how much of the previous hidden state to forget when calculating the candidate.

Candidate hidden state

$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h)$$

Hidden state update

$$h_t = (1 - z_t) \odot \tilde{h}_t + z_t \odot h_{t-1}$$

Interpretation:

- z_t acts like both forget & input gates combined
- r_t controls contribution of h_{t-1} to new content
- $\tilde{h}_t$ is the proposed hidden state
- h_t mixes old and new information

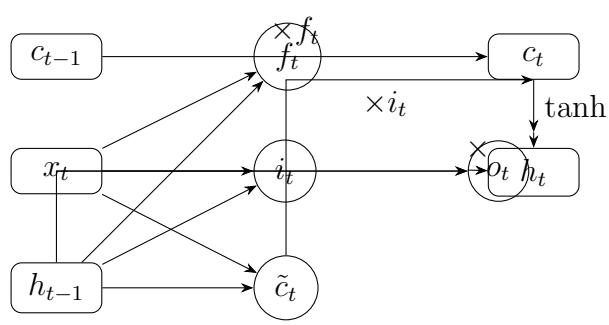
8.3 Unified Comparison and Insights

- **Vanilla RNN:** No gates; memory updated entirely by nonlinear recurrence.
- **GRU:** Two gates for coarse memory control.
- **LSTM:** Three gates + separate cell state for fine-grained long-term memory management.

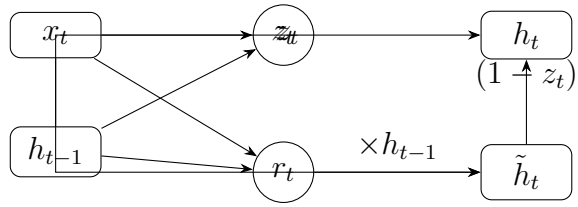
LSTMs use three gates and a separate cell state to control memory, while GRUs use only two gates and merge the cell state with the hidden state for a simpler, faster architecture. All gated architectures enhance the RNN's ability to handle long-range dependencies, but LSTM provides the most expressive control via its three gates and independent cell state.

8.4 Diagrams for LSTM and GRU Gates

8.4.1 LSTM Gate Diagram



8.4.2 GRU Gate Diagram



8.5 Worked Numerical Example: LSTM Gates

Consider a scalar LSTM cell with the following parameters:

$$x_t = 1.0, \quad h_{t-1} = 0.5, \quad c_{t-1} = 0.1,$$

$$W_f = 0.5,\ U_f = 0.1,\ b_f = 0, \quad W_i = 0.4,\ U_i = 0.2,\ b_i = 0,$$
$$W_c = 0.6,\ U_c = 0.1,\ b_c = 0, \quad W_o = 0.3,\ U_o = 0.2,\ b_o = 0.$$

Let $\sigma(\cdot)$ denote the logistic sigmoid function and $\tanh(\cdot)$ the hyperbolic tangent.

Step 1: Forget gate.

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f) = \sigma(0.5 \cdot 1.0 + 0.1 \cdot 0.5) = \sigma(0.55) \approx 0.634.$$

Step 2: Input gate.

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i) = \sigma(0.4 \cdot 1.0 + 0.2 \cdot 0.5) = \sigma(0.5) \approx 0.622.$$

Step 3: Candidate cell state.

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c) = \tanh(0.6 \cdot 1.0 + 0.1 \cdot 0.5) = \tanh(0.65) \approx 0.572.$$

Step 4: Output gate.

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o) = \sigma(0.3 \cdot 1.0 + 0.2 \cdot 0.5) = \sigma(0.4) \approx 0.599.$$

Step 5: New cell state.

$$c_t = f_t c_{t-1} + i_t \tilde{c}_t \approx 0.634 \cdot 0.1 + 0.622 \cdot 0.572 \approx 0.0634 + 0.3559 \approx 0.4193.$$

Step 6: New hidden state.

$$h_t = o_t \tanh(c_t) \approx 0.599 \cdot \tanh(0.4193) \approx 0.599 \cdot 0.397 \approx 0.237.$$

Thus, from a single time step we obtain:

$$f_t \approx 0.63, \quad i_t \approx 0.62, \quad \tilde{c}_t \approx 0.57, \quad o_t \approx 0.60, \quad c_t \approx 0.42, \quad h_t \approx 0.24.$$

8.6 Worked Numerical Example: GRU Gates

Consider a scalar GRU cell with:

$$x_t = 1.0, \quad h_{t-1} = 0.5,$$

$$W_z = 0.4,\ U_z = 0.1,\ b_z = 0, \quad W_r = 0.3,\ U_r = 0.2,\ b_r = 0,$$
$$W_h = 0.5,\ U_h = 0.1,\ b_h = 0.$$

Step 1: Update gate.

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z) = \sigma(0.4 \cdot 1.0 + 0.1 \cdot 0.5) = \sigma(0.45) \approx 0.611.$$

Step 2: Reset gate.

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r) = \sigma(0.3 \cdot 1.0 + 0.2 \cdot 0.5) = \sigma(0.4) \approx 0.599.$$

Step 3: Candidate hidden state.

$$\tilde{h}_t = \tanh\big(W_h x_t + U_h(r_t h_{t-1}) + b_h\big) = \tanh\big(0.5 \cdot 1.0 + 0.1 \cdot (0.599 \cdot 0.5)\big)$$
$$\approx \tanh(0.5 + 0.02995) = \tanh(0.530) \approx 0.485.$$

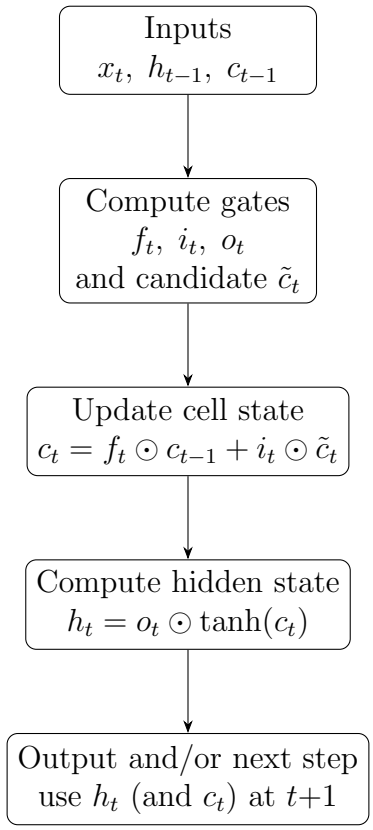
Step 4: New hidden state.

$$h_t = (1 - z_t)\tilde{h}_t + z_t h_{t-1} \approx (1 - 0.611) \cdot 0.485 + 0.611 \cdot 0.5$$
$$\approx 0.389 \cdot 0.485 + 0.611 \cdot 0.5 \approx 0.189 + 0.306 \approx 0.494.$$

So the GRU produces:

$$z_t \approx 0.61, \quad r_t \approx 0.60, \quad \tilde{h}_t \approx 0.49, \quad h_t \approx 0.49.$$

8.7 Flowchart: "How information flows through each gate" (LSTM)



This flowchart summarizes how information moves through the LSTM: inputs feed the gates, the gates modulate how the cell state is updated, and the output gate controls how much of the updated cell state contributes to the new hidden state and subsequent outputs.

9 Recurrent Neural Networks: Questions and Solutions

Conceptual Questions

1. Explain in your own words why RNNs are suitable for sequential data compared to feedforward networks.

   Solution. Feedforward networks (FFNNs) treat each input independently: the output for x_t depends only on x_t and fixed weights. They have no built-in notion of order or memory. Recurrent neural networks (RNNs) introduce a hidden state h_t that is updated as

   $$h_t = \phi(W_x x_t + W_h h_{t-1} + b),$$

   so the current hidden state depends both on the current input x_t and the previous hidden state h_{t-1}. Unrolling this over time means h_t carries information from all past inputs $x_1, \ldots, x_t$. This makes RNNs naturally suited to sequential data where order

matters (text, speech, time series), because they can model temporal dependencies and context.

2. Draw an unrolled RNN for three time steps and label x_1, x_2, x_3, the hidden states h_1, h_2, h_3, and the outputs y_1, y_2, y_3.

   Solution. (Description for notes; you can keep your TikZ diagram here.) An unrolled RNN for $T = 3$ has:

$$x_1 \to h_1 \to y_1, \quad x_2 \to h_2 \to y_2, \quad x_3 \to h_3 \to y_3,$$

   with recurrent connections

$$h_1 \to h_2 \to h_3.$$

   More explicitly:

$$\begin{aligned} h_1 &= \phi(W_x x_1 + W_h h_0 + b), \quad & y_1 &= W_y h_1 + b_y, \\ h_2 &= \phi(W_x x_2 + W_h h_1 + b), \quad & y_2 &= W_y h_2 + b_y, \\ h_3 &= \phi(W_x x_3 + W_h h_2 + b), \quad & y_3 &= W_y h_3 + b_y. \end{aligned}$$

3. Define saturation in the context of RNN activations. Why does saturation cause difficulties during training?

   Solution. With activation functions such as tanh or σ, *saturation* occurs when the input to the nonlinearity is very large in magnitude (positive or negative), so the output is close to its asymptotic value (± 1 for tanh, 0 or 1 for sigmoid). In these saturated regions, the derivative is very close to zero:

$$\frac{d}{du} \tanh(u) = 1 - \tanh^2(u) \approx 0 \quad \text{when } |u| \text{ is large.}$$

   During backpropagation, gradients are multiplied by these derivatives at each time step. If many time steps are saturated, these near-zero derivatives cause the gradients to shrink towards zero, leading to vanishing gradients and making it very hard for the RNN to learn long-range dependencies.

4. Consider the RNN update equation:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b).$$

   Substitute recursively to express h_3 in terms of x_1, x_2, x_3 and the weights.

   Solution. First write:

$$h_1 = \tanh(W_x x_1 + W_h h_0 + b).$$

   Then

$$h_2 = \tanh\big(W_x x_2 + W_h h_1 + b\big) = \tanh\big(W_x x_2 + W_h \tanh(W_x x_1 + W_h h_0 + b) + b\big).$$

   Finally,

$$\begin{aligned} h_3 &= \tanh\big(W_x x_3 + W_h h_2 + b\big) \\ &= \tanh\Big(W_x x_3 + W_h \tanh\big(W_x x_2 + W_h \tanh(W_x x_1 + W_h h_0 + b) + b\big) + b\Big). \end{aligned}$$

   This shows explicitly how h_3 is a nested nonlinear function of x_1, x_2, x_3 and the weights (and initial state h_0).

10 Backpropagation Through Time (BPTT)

1. Show that the derivative of h_t with respect to h_{t-k} is:

$$\frac{\partial h_t}{\partial h_{t-k}} = \prod_{i=t-k+1}^{t} W_h^\top \left(1 - h_i^2\right).$$

Solution. For a vanilla RNN

$$h_i = \tanh(W_x x_i + W_h h_{i-1} + b),$$

we can write the Jacobian of h_i w.r.t. h_{i-1} as

$$\frac{\partial h_i}{\partial h_{i-1}} = \underbrace{\mathrm{diag}(1 - h_i^2)}_{\tanh'(\cdot)} W_h.$$

If we look at the transpose (as is common in gradient flow),

$$\left(\frac{\partial h_i}{\partial h_{i-1}}\right)^\top = W_h^\top \,\mathrm{diag}(1 - h_i^2).$$

By the chain rule over k time steps:

$$\frac{\partial h_t}{\partial h_{t-k}} = \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial h_{t-2}} \cdots \frac{\partial h_{t-k+1}}{\partial h_{t-k}}.$$

Taking the transpose form (or equivalently multiplying in the appropriate order) gives

$$\frac{\partial h_t}{\partial h_{t-k}} = \prod_{i=t-k+1}^{t} W_h^\top \left(1 - h_i^2\right),$$

where $(1-h_i^2)$ is understood as the vector of derivatives, often represented as a diagonal matrix.

2. Explain:

   (a) If $\|W_h\| < 1$, what happens to the gradient?

   (b) If $\|W_h\| > 1$, what happens?

   (c) Why does this lead to vanishing or exploding gradients?

Solution. Using the expression above, the gradient flowing back k steps involves a product of k terms approximately on the order of $\|W_h\|$ times derivatives $\approx 1 - h_i^2$ (often bounded by 1 in magnitude).

   (a) If $\|W_h\| < 1$, then each multiplication tends to shrink the gradient. After k steps, the norm behaves roughly like $\|W_h\|^k$, which decays exponentially to zero as k grows. This yields *vanishing gradients*.

(b) If $\|W_h\| > 1$, then each multiplication tends to magnify the gradient; after many steps the gradient can grow exponentially, leading to *exploding gradients*.

(c) In both cases, the repeated multiplication by $W_h^\top$ and activation derivatives causes the backpropagated signal to either decay to zero (vanish) or blow up to very large values (explode). This makes it difficult for gradient-based optimization to learn long-range dependencies in vanilla RNNs.

11 Numerical RNN Exercises

1. Given:

$$x_t = 1.2, \quad h_{t-1} = 0.5, \quad W_x = 1.0, \quad W_h = 0.8, \quad b = 0,$$

compute:

$$u_t = W_x x_t + W_h h_{t-1}, \qquad h_t = \tanh(u_t).$$

Solution.

$$u_t = 1.0 \cdot 1.2 + 0.8 \cdot 0.5 = 1.2 + 0.4 = 1.6.$$

Then

$$h_t = \tanh(1.6) \approx 0.922 \quad \text{(approximately)}.$$

2. Suppose:

$$\frac{\partial L}{\partial h_T} = 1, \qquad \frac{\partial h_t}{\partial h_{t-1}} = 1.2.$$

Compute:

$$\frac{\partial L}{\partial h_{T-10}}.$$

Comment on exploding gradients.

Solution. Over 10 steps:

$$\frac{\partial L}{\partial h_{T-10}} = \frac{\partial L}{\partial h_T} \prod_{k=0}^{9} \frac{\partial h_{T-k}}{\partial h_{T-k-1}} = 1 \cdot (1.2)^{10} \approx 6.19.$$

The gradient has grown more than six-fold in just 10 steps. For much longer sequences the growth is exponential, which is an example of *exploding gradients*.

3. Repeat the previous question with 0.5 instead of 1.2 and discuss vanishing gradients.

Solution. Now each step multiplies the gradient by 0.5:

$$\frac{\partial L}{\partial h_{T-10}} = 1 \cdot (0.5)^{10} = \frac{1}{1024} \approx 0.00098.$$

After just 10 steps, the gradient is almost zero. For longer sequences, it becomes numerically negligible, illustrating the *vanishing gradient* problem.

Conceptual Questions

1. Match each role to the correct gate: (a) Controls forgetting (b) Controls update strength (c) Candidate new memory (d) Controls exposure to output

   Choose from: f_t, i_t, $\tilde{c}_t$, o_t.

   Solution.

   - (a) Controls forgetting $\Rightarrow f_t$ (forget gate)
   - (b) Controls update strength $\Rightarrow i_t$ (input gate)
   - (c) Candidate new memory $\Rightarrow \tilde{c}_t$ (candidate cell content)
   - (d) Controls exposure to output $\Rightarrow o_t$ (output gate)

2. Explain in a short paragraph why LSTMs solve the vanishing gradient problem. Use:

$$\frac{\partial c_t}{\partial c_{t-1}} = f_t.$$

   Solution. In an LSTM, the cell state update is

$$c_t = f_t c_{t-1} + i_t \tilde{c}_t.$$

   The derivative of c_t w.r.t. c_{t-1} is simply

$$\frac{\partial c_t}{\partial c_{t-1}} = f_t,$$

   which is a gate value typically in $(0, 1)$. Over multiple time steps,

$$\frac{\partial c_t}{\partial c_{t-k}} = \prod_{i=t-k+1}^{t} f_i.$$

   If f_i stays close to 1, the product does not vanish, so gradients can flow backwards across many time steps. This "linear" path for error flow through c_t (the Constant Error Carousel when $f_i = 1$) prevents the severe vanishing gradients seen in vanilla RNNs, allowing LSTMs to capture long-term dependencies.

LSTM Numerical Problems

1. Given:

$$x_t = 0.5, \quad h_{t-1} = 0.2, \quad c_{t-1} = 0.1,$$

   with weights:

$$W_f = 2,\ U_f = -1; \quad W_i = 1,\ U_i = 1; \quad W_c = 1,\ U_c = 0; \quad W_o = -1,\ U_o = 2,$$

   compute:

$$f_t,\ i_t,\ \tilde{c}_t,\ c_t,\ o_t,\ h_t.$$

Use $\sigma(s) = \frac{1}{1+e^{-s}}$ and $\tanh(s)$.

Solution. First compute the pre-activations:

Forget gate:

$$s_f = W_f x_t + U_f h_{t-1} = 2 \cdot 0.5 + (-1) \cdot 0.2 = 1.0 - 0.2 = 0.8,$$

so

$$f_t = \sigma(0.8) \approx 0.690.$$

Input gate:

$$s_i = W_i x_t + U_i h_{t-1} = 1 \cdot 0.5 + 1 \cdot 0.2 = 0.7,$$

so

$$i_t = \sigma(0.7) \approx 0.668.$$

Candidate cell:

$$s_c = W_c x_t + U_c h_{t-1} = 1 \cdot 0.5 + 0 \cdot 0.2 = 0.5,$$

so

$$\tilde{c}_t = \tanh(0.5) \approx 0.462.$$

Cell update:

$$\begin{aligned} c_t &= f_t c_{t-1} + i_t \tilde{c}_t \\ &\approx 0.690 \cdot 0.1 + 0.668 \cdot 0.462 \\ &\approx 0.069 + 0.309 \approx 0.378. \end{aligned}$$

Output gate:

$$s_o = W_o x_t + U_o h_{t-1} = -1 \cdot 0.5 + 2 \cdot 0.2 = -0.5 + 0.4 = -0.1,$$

so

$$o_t = \sigma(-0.1) \approx 0.475.$$

Hidden state:

$$h_t = o_t \tanh(c_t) \approx 0.475 \cdot \tanh(0.378) \approx 0.475 \cdot 0.361 \approx 0.171.$$

So approximately:

$$f_t \approx 0.690, \quad i_t \approx 0.668, \quad \tilde{c}_t \approx 0.462, \quad c_t \approx 0.378, \quad o_t \approx 0.475, \quad h_t \approx 0.171.$$

LSTM Derivation Problems

1. Show that:

$$\frac{\partial c_t}{\partial c_{t-k}} = \prod_{i=t-k+1}^{t} f_i.$$

Solution. The LSTM cell update is

$$c_t = f_t c_{t-1} + i_t \tilde{c}_t,$$

so

$$\frac{\partial c_t}{\partial c_{t-1}} = f_t.$$

Similarly,

$$\frac{\partial c_{t-1}}{\partial c_{t-2}} = f_{t-1}, \quad \ldots, \quad \frac{\partial c_{t-k+1}}{\partial c_{t-k}} = f_{t-k+1}.$$

By the chain rule:

$$\frac{\partial c_t}{\partial c_{t-k}} = \frac{\partial c_t}{\partial c_{t-1}} \frac{\partial c_{t-1}}{\partial c_{t-2}} \cdots \frac{\partial c_{t-k+1}}{\partial c_{t-k}} = \prod_{i=t-k+1}^{t} f_i.$$

2. Discuss three cases:

   (a) $f_i = 0.9$ for all i

   (b) $f_i = 0.1$ for all i

   (c) $f_i = 1$ for all i (Constant Error Carousel)

   Solution.

   (a) If $f_i = 0.9$ for all i, then

   $$\frac{\partial c_t}{\partial c_{t-k}} = 0.9^k,$$

   which decays exponentially but fairly slowly. Gradients diminish with time but can still persist over a moderate number of steps.

   (b) If $f_i = 0.1$ for all i, then

   $$\frac{\partial c_t}{\partial c_{t-k}} = 0.1^k,$$

   which vanishes extremely quickly. Information is effectively forgotten after only a few time steps.

   (c) If $f_i = 1$ for all i, then

   $$\frac{\partial c_t}{\partial c_{t-k}} = 1^k = 1,$$

   so the gradient is preserved perfectly across time. This is the Constant Error Carousel (CEC), in which error signals circulate without decay, allowing the LSTM to store information over very long sequences.

3. Explain mathematically why the additive update:

$$c_t = f_t c_{t-1} + i_t \tilde{c}_t$$

is more stable than the RNN update:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t).$$

Solution. In the vanilla RNN, the hidden state update is

$$h_t = \tanh(W_h h_{t-1} + \cdots),$$

so

$$\frac{\partial h_t}{\partial h_{t-1}} = \operatorname{diag}(1 - h_t^2)\, W_h.$$

Backpropagated gradients are products of these matrices. The combination of W_h and the derivative of tanh often leads to norms either < 1 (vanishing) or > 1 (exploding).

In contrast, the LSTM cell state has an *additive* update:

$$c_t = f_t c_{t-1} + i_t \tilde{c}_t,$$

with

$$\frac{\partial c_t}{\partial c_{t-1}} = f_t,$$

and backpropagation along c_t involves products of scalars (or diagonal entries) f_i that can be explicitly controlled by the network. When $f_i \approx 1$, gradients remain stable across many time steps. The key stability difference is that the LSTM's memory path is linear (additive) with gated scaling, while the RNN's memory path is fully embedded inside a nonlinear transformation and a recurrent weight matrix, making its gradient behavior much harder to control.

Additional RNN / LSTM / GRU Exercises

1. **RNN Forward Computation with Mixed Activations**

   Consider an RNN with

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h), \qquad y_t = \sigma(W_y h_t + b_y).$$

   Given:

$$h_0 = 0, \quad x_1 = 1.5,\ x_2 = -0.8,$$
$$W_x = 0.7,\ W_h = -0.3,\ W_y = 2.0, \quad b_h = 0.1,\ b_y = -1.$$

   Compute h_1, h_2, y_1, y_2.

2. **Identifying Sequence Type**

   For each task, determine whether the appropriate architecture is: One-to-One, One-to-Many, Many-to-One, Many-to-Many (Aligned), or Many-to-Many (Encoder–Decoder).

   (a) Predicting the next 5 video frames from 1 input frame.

   (b) Speech-to-text transcription.

   (c) Rating sentiment of a full movie review.

   (d) Generating drum patterns from a single style vector.

   (e) POS tagging of a sentence.

3. **BPTT Gradient Behavior**

   Given:

$$\frac{\partial h_t}{\partial h_{t-1}} = 0.92, \qquad \frac{\partial L}{\partial h_T} = 0.8,$$

   compute the symbolic form of

$$\frac{\partial L}{\partial h_{T-20}}.$$

   Explain whether vanishing or exploding gradients occur.

4. **Forget Gate Interpretation in LSTM**

   Given:

t	f_t	i_t	$\tilde{c}_t$
1	0.95	0.20	0.10
2	0.10	0.90	0.80
3	0.70	0.60	0.30

   Let $c_0 = 0.5$. Compute c_1, c_2, c_3. Indicate at which step most forgetting occurs and at which step memory is updated most strongly.

5. **GRU Candidate Activation**

   Given the GRU equations:

$$z_t = \sigma(W_z x_t + U_z h_{t-1}), \qquad r_t = \sigma(W_r x_t + U_r h_{t-1}),$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1})), \qquad h_t = (1 - z_t)\tilde{h}_t + z_t h_{t-1},$$

   compute $\tilde{h}_t$ and h_t for:

$$x_t = 0.6,\ h_{t-1} = 0.4,$$

$$W_z = 0.8,\ U_z = 0.2, \quad W_r = -0.4,\ U_r = 0.3,$$

$$W_h = 1.2,\ U_h = -0.5.$$

6. **RNN vs LSTM Performance**

   Give two reasons why an RNN and an LSTM behave differently on a long-term seasonal prediction task (e.g., rainfall forecasting). Discuss in terms of gradient flow, memory mechanisms, and activation saturation.

7. **Gradient Norm Explosion**

   Consider the recurrent matrix:

$$W_h = \begin{pmatrix} 1.4 & 0.1 & 0 \\ 0 & 1.2 & 0.2 \\ 0 & 0 & 1.3 \end{pmatrix}.$$

   Explain whether exploding gradients are likely during BPTT. Describe how gradient clipping addresses the issue.

8. **LSTM Gate Reasoning**

   For each scenario, describe the qualitative behavior of f_t, i_t, and o_t:

   (a) A strongly emotional word appears (e.g. "excellent").

   (b) A filler word appears (e.g. "um", "uh").

   (c) A sentiment reversal occurs (e.g. "not good").

   Describe what the memory cell should do in each case.

9. **Parameter Counting in LSTM**

   An LSTM has input dimension 5 and hidden dimension 4. Each gate has:

$$W \in \mathbb{R}^{4\times 5}, \quad U \in \mathbb{R}^{4\times 4}, \quad b \in \mathbb{R}^4.$$

   Compute the total number of trainable parameters for all four gates combined.

10. **LSTM vs GRU: Practical Comparison**

    A claim is made:

    > "GRUs always outperform LSTMs because they have fewer parameters."

    Discuss why this is not necessarily true. Compare gating structure, model capacity, data requirements, and long-term dependency modeling.

Further LSTM Gate-Control Numerical Exercises

1. **Exercise 1**

   Suppose:
   $$c_{t-1} = 0.40, \quad \tilde{c}_t = 0.90,$$
   and gates:
   $$f_t = 0.20, \qquad i_t = 0.80.$$
   Compute the new cell state c_t using
   $$c_t = f_t c_{t-1} + i_t \tilde{c}_t.$$
   Briefly interpret what the LSTM is doing with old vs new information.

2. **Exercise 2**

   Suppose:
   $$c_{t-1} = -0.30, \quad \tilde{c}_t = 0.50,$$
   and:
   $$f_t = 0.90, \qquad i_t = 0.10.$$
   Compute c_t and explain whether the cell mostly keeps old memory or writes new memory.

3. **Exercise 3**

   Suppose:
   $$c_{t-1} = 1.00, \quad \tilde{c}_t = -0.40,$$
   and gates:
   $$f_t = 0.70, \qquad i_t = 0.30.$$
   Compute c_t. Is the cell state moving closer to zero, further away, or changing sign?

4. **Exercise 4**

   Suppose:
   $$c_{t-1} = 0.10, \quad \tilde{c}_t = 0.90,$$
   with:
   $$f_t = 0.05, \qquad i_t = 0.95.$$
   Compute c_t. Interpret what happens when the forget gate is very small and the input gate is very large.

5. **Exercise 5**

   Suppose:
   $$c_{t-1} = -0.80, \quad \tilde{c}_t = -0.20,$$
   and gates:
   $$f_t = 0.60, \qquad i_t = 0.40.$$
   Compute c_t. Comment on whether the LSTM is reinforcing or weakening the negative memory.

6. **Exercise 6**

   An LSTM cell has:
   $$c_{t-1} = 0.75, \quad \tilde{c}_t = 0.10,$$
   and:
   $$f_t = 0.95, \qquad i_t = 0.05.$$
   Compute c_t. Explain why this setting is appropriate if the past information is very important and the new signal is weak.

7. **Exercise 7**

   Suppose:
   $$c_{t-1} = -0.50, \quad \tilde{c}_t = 0.80,$$
   with:
   $$f_t = 0.40, \qquad i_t = 0.90.$$
   Compute c_t. Does the new cell state mainly reflect the old negative value or the new positive value?

8. **Exercise 8**

   For:
   $$c_{t-1} = 0.00, \quad \tilde{c}_t = 1.00,$$
   assume:
   $$f_t = 1.00, \qquad i_t = 0.00.$$
   Compute c_t. Interpret what this means in terms of "blocking" update.

9. **Exercise 9**

   Suppose:
   $$c_{t-1} = 0.60, \quad \tilde{c}_t = -0.60,$$
   and gates:
   $$f_t = 0.50, \qquad i_t = 0.50.$$
   Compute c_t. Discuss whether the LSTM is moving towards forgetting, reinforcing, or cancelling information.

10. **Exercise 10**

Consider:

$$c_{t-1} = -0.20, \quad \tilde{c}_t = -0.90,$$

and:

$$f_t = 0.85, \qquad i_t = 0.15.$$

Compute c_t. Explain a realistic NLP scenario (e.g., sentiment analysis) in which such gate values might be useful.

References

[1] Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536.

[2] Werbos, P. J. (1990). Backpropagation Through Time: What It Does and How to Do It. *Proceedings of the IEEE*, 78(10), 1550–1560.

[3] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.

[4] Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation. *Proceedings of EMNLP 2014*.

[5] Chung, J., Gulcehre, C., Cho, K., & Bengio, Y. (2014). Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling. arXiv:1412.3555.

[6] Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to Sequence Learning with Neural Networks. *Proceedings of NeurIPS 2014*.

[7] Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. *Proceedings of ICLR 2015*.

[8] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. *Proceedings of NeurIPS 2017*.

[9] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.

[10] Zhang, A., Lipton, Z. C., Li, M., & Smola, A. (2020). *Dive into Deep Learning*. Available online.

[11] Nielsen, M. A. (2015). *Neural Networks and Deep Learning*. Online book.

[12] Greff, K., Srivastava, R. K., Koutník, J., Steunebrink, B. R., & Schmidhuber, J. (2017). LSTM: A Search Space Odyssey. *IEEE Transactions on Neural Networks and Learning Systems*, 28(10), 2222–2232.

[13] Lipton, Z. C. (2015). A Critical Review of Recurrent Neural Networks for Sequence Learning. arXiv:1506.00019.

[14] Graves, A., Mohamed, A., & Hinton, G. (2013). Speech Recognition with Deep Recurrent Neural Networks. *Proceedings of ICASSP 2013*.

[15] Schuster, M., & Paliwal, K. K. (1997). Bidirectional Recurrent Neural Networks. *IEEE Transactions on Signal Processing*, 45(11), 2673–2681.

[16] Oladeji, T. A., Adeyemo, B. T., & Awe, O. O. (2024). Predicting Precipitation Time Series Dynamics in West Africa Using Deep Learning Models. In *Practical Statistical Learning and Data Science Methods: Case Studies from LISA 2020 Global Network, USA* (pp. 111–125). Cham: Springer Nature Switzerland.